Coordination Architecture for AI Coding Agent Harnesses

First Author[1], Second Author[2], Third Author[2], Fourth Author[1] and Fifth Author[1]
[*]Equal Contribution, [1]Affiliation 1, [2]Affiliation 2

Multi-agent systems for software engineering promise throughput gains through parallelism, but naive scaling amplifies errors by up to 17.2× and can produce net-negative quality outcomes. This paper develops a formal framework for reasoning about coordination architecture in multi-agent code generation. We make four principal contributions: (1) an error amplification model showing that the standard independence assumption predicts amplification approaching k from below, dramatically underestimating the empirical $17.2\times$ observed by Kim et al. (2025), implying that agent errors are correlated and cascading; (2) a formulation of task decomposition as balanced min-cut on file-coupling hypergraphs, connecting partition quality to merge conflict rate via a Cut-Conflict Bridge lemma; (3) a five-level merge conflict taxonomy with a probabilistic graphical model showing $O(k^2)$ growth in uncoordinated systems and $O(k)$ reduction under file-ownership contracts; (4) a Quality-Adjusted Speedup (QAS) metric demonstrating that parallelism can yield QAS < 1 (worse than sequential execution) when coordination quality is insufficient. We ground these results in empirical data from Kim et al.'s 180-configuration scaling study, DORA 2025 telemetry from 10,000+ developers, and Cursor's production experience with hundreds of concurrent agents. We formalize coordination as concurrency control, map database isolation levels to agent mechanisms, develop assume-guarantee contracts for compositional correctness, and express Conway's Law as a communication-graph containment theorem. Six design principles and five verification experiments are proposed.

1. Introduction

The deployment of large language models (LLMs) as autonomous software engineering agents has progressed from single-agent assistants to multi-agent systems where multiple LLM instances work concurrently on a shared codebase. This parallelization promises dramatic throughput improvements: Cursor's production system ran hundreds of agents simultaneously for weeks, producing over one million lines of code for a web browser implementation (Cursor, 2026). Amazon Q Developer deploys coordinated agent teams for parallel development (Osmani, 2025).

Yet the empirical evidence paints a more nuanced picture. Kim et al. (2025) conducted the most comprehensive evaluation to date (180 configurations across 5 architectures, 3 LLM families, and 4 benchmarks), finding that independent agents amplify errors by 17.2× relative to a single agent, while even centralized coordination still amplifies errors 4.4×. Google's DORA 2025 report (DORA Team, Google Cloud, 2025), drawing on telemetry from over 10,000 developers, found that AI adoption increased merged pull requests by 98% but also increased code review time by 91%, PR size by 154%, and bug rates by 9%, with organizational delivery metrics remaining flat. Cursor (2026) reported (in a practitioner blog post, not a peer-reviewed study) that a flat coordination model with 20 agents collapsed to effective throughput of 2 to 3 agents due to lock contention.

These findings reveal a fundamental tension: parallelism increases throughput but also increases error rate, and the two effects work against each other. The question is not whether to parallelize, but how to parallelize safely, and when not to parallelize at all.

This paper develops a formal framework for reasoning about these tradeoffs. Our contributions are:

(1) An **error amplification analysis** (Section 3) showing that the standard independence model predicts amplification of at most k, dramatically underestimating the empirical $17.2\times$ from Kim et al. (2025). The $\approx 6\times$ discrepancy quantifies the "correlation multiplier" from inter-agent error propagation. We extend Amdahl's Law with a reliability factor and derive the optimal agent count $n^* \approx 1/\sqrt{\delta}$.
(2) A **task decomposition framework** (Section 4) formulating the problem as balanced min-cut on file-coupling hypergraphs, connecting partition quality to merge conflict rate via a Cut-Conflict Bridge lemma, and surveying the algorithmic landscape (spectral methods, METIS, FM) with their approximation guarantees.
(3) A **five-level merge conflict taxonomy** (Section 5) with a probabilistic graphical model showing $O(k^2)$ conflict growth without contracts and $O(k)$ growth with file-ownership contracts, grounded in empirical conflict rates from studies spanning thousands of open-source projects.
(4) A **Quality-Adjusted Speedup** metric (Section 8) that accounts for both throughput gains and quality costs, demonstrating regimes where QAS < 1 (parallelism is net-negative), calibrated against DORA and Kim et al. data.

Additionally, we formalize coordination as concurrency control (Section 6), develop a topology selection framework with dominance conditions (Section 7), express Conway's Law as a graph containment theorem (Section 7), and derive six actionable design principles (Section 9).

This is a theory and position paper. We develop formal models grounded in existing empirical data, but we do not present new experimental results. We are explicit throughout about which claims are established theory, which are novel synthesis, and which are conjectures awaiting verification. Five calibration experiments are proposed in Section 11.

2. Preliminaries and Definitions

We establish the core definitions used throughout the paper.

Definition 2.1 (Coupling Graph)**.** *[Established] Given a codebase with file set $\mathcal{F} = \{f_1, \ldots, f_n\}$, the **coupling graph** is an undirected weighted graph $G = (F, E, w)$ where $F = \mathcal{F}$, the edge set E encodes coupling relationships (structural imports, co-change frequency from version history, semantic similarity), and the weight function $w : E \to \mathbb{R}_+$ encodes coupling strength as a composite metric.*

The composite weight combines three empirically validated coupling dimensions (Accioly et al., 2018; Kasi and Sarma, 2013; Martin, 1994): structural coupling (import/call-graph edges), co-change coupling (files frequently committed together in version history), and semantic coupling (identifier and comment similarity). Co-change coupling is the strongest single predictor of merge conflicts; code associated with merge conflicts is $2\times$ more likely to contain bugs, rising to $26\times$ when conflicts require manual resolution (Accioly et al., 2018).

Definition 2.2 (Error Amplification Factor)**.** *[Synthesis] For coordination topology $t \in \mathcal{T}$ with k agents, each having per-agent error rate $\epsilon = 1 - p_{\text{single}}$, the **error amplification factor** is:*

$$A_e(t, k) = \frac{P(\textit{system error} \mid t, k)}{P(\textit{single-agent error})} = \frac{1 - R_{\text{sys}}(t, k)}{\epsilon} \tag{1}$$

where $R_{\text{sys}}(t, k)$ is the system reliability (probability that all agent outputs are jointly correct).

Definition 2.3 (Topology Space). *[Synthesis] The set of coordination topologies is* $\mathcal{T} = \{\text{indep}, \text{central}, \text{hier}, \text{iso-merg}$ *representing independent (no coordination), centralized (hub-and-spoke), hierarchical (tree), and isolated-then-merge (worktree isolation with deferred integration) architectures.*

Definition 2.4 (Coordination Contract). *[Synthesis] A* ***coordination contract*** *for k agents on codebase $\mathcal{F}$ is a tuple $C = (\{F_i\}_{i=1}^{k}, \{I_j\}_{j=1}^{m}, \sigma)$ where:*

- *$F_i \subseteq \mathcal{F}$ is the owned file set of agent A_i, with $F_i \cap F_j = \emptyset$ for $i \neq j$;*
- *$\{I_j\}_{j=1}^{m}$ is a set of interface specifications, each $I_j = (\text{name}_j, \text{sig}_j, \text{module}_j)$;*
- *$\sigma : \mathcal{F} \to 2^{\{I_1,\ldots,I_m\}}$ maps each file to its exported interfaces.*

Definition 2.5 (Isolation Levels for Agents). *[Synthesis] We define four isolation levels for multi-agent code generation, each corresponding to a database isolation level (Berenson et al., 1995):*

- ReadUncommitted*: shared workspace, no branches; all anomalies permitted.*
- ReadCommitted*: git branches with periodic pulls; prevents dirty reads.*
- Snapshot*: git worktrees frozen at dispatch; prevents dirty reads, non-repeatable reads, and phantoms; permits write skew.*
- Serializable*: sequential execution; prevents all anomalies.*

3. Error Amplification and Scaling Limits

3.1. Error Amplification under Independence

The simplest model assumes agents produce errors independently with probability ϵ, and the system requires all k outputs to be jointly correct (the "series reliability" model).

Theorem 3.1 (Error Amplification under Independence). *[Established] + [Synthesis] For k independent agents in serial composition, the system reliability is $R_{\text{sys}}(\text{indep}, k) = (1-\epsilon)^k$ and the error amplification factor is:*

$$A_e(\text{indep}, k) = \frac{1-(1-\epsilon)^k}{\epsilon} \tag{2}$$

This quantity is strictly increasing in k, satisfies $A_e(\text{indep}, 1) = 1$, and admits the expansion:

$$A_e(\text{indep}, k) = k - \binom{k}{2}\epsilon + O(\epsilon^2) \tag{3}$$

Thus A_e approaches k from below as $\epsilon \to 0$, with deficit $\binom{k}{2}\epsilon$. Note that the two-term expansion is accurate only for small ϵ; at practical error rates ($\epsilon \approx 0.2$-0.3), higher-order terms are not negligible and the exact formula (2) should be used.

Proof. The system error probability is $1-(1-\epsilon)^k$, giving $A_e = (1-(1-\epsilon)^k)/\epsilon$. Expanding $(1-\epsilon)^k$ via the binomial series: $(1-\epsilon)^k = 1-k\epsilon+\binom{k}{2}\epsilon^2-\cdots$, so $1-(1-\epsilon)^k = k\epsilon-\binom{k}{2}\epsilon^2+O(\epsilon^3)$, and dividing by ϵ yields $A_e = k-\binom{k}{2}\epsilon+O(\epsilon^2)$. Monotonicity in k follows from the derivative: $\partial A_e/\partial k = -(1-\epsilon)^k \ln(1-\epsilon)/\epsilon > 0$ for $\epsilon \in (0,1)$. □

Remark. *Kim et al. (2025) measured $A_e(\text{indep}) = 17.2$ with 95% CI $[14.3, 20.1]$. For $k = 5$ agents at $\epsilon = 0.3$, the independence model gives $A_e = (1-0.7^5)/0.3 \approx 2.77$, far below the empirical value. This discrepancy implies that agent errors are correlated and cascading: an error by one agent propagates to others through shared assumptions, producing compounding failures that the independence model cannot capture. The factor of $\approx 6\times$ between the model and observation quantifies the "correlation multiplier" attributable to inter-agent error propagation.*

3.2. Information-Sharing Reduction Factor

Definition 3.1 (Coordination Benefit Function)**.** *[Synthesis] For topology t with information-sharing capacity $I(t)$ bits per coordination round, define the effective per-agent error rate under coordination:*

$$\epsilon_t = \epsilon \cdot g(I(t)) \tag{4}$$

where $g : \mathbb{R}_+ \to (0, 1]$ is monotonically decreasing with $g(0) = 1$ and $\lim_{I\to\infty} g(I) = g_{\min} > 0$ (coordination cannot eliminate intrinsic errors).

A natural choice is $g(I) = e^{-\alpha I}$ for topology-dependent rate $\alpha > 0$. Under this model, the ratio of centralized to independent error amplification calibrates α: from Kim et al. (2025), $4.4/17.2 \approx 0.256$, giving $\alpha \cdot I(\text{central}) \approx 1.36$ nats.

3.3. Capability Saturation Crossover

Definition 3.2 (Saturation Threshold)**.** *[Synthesis] The **capability saturation threshold** P^* is the single-agent accuracy at which the marginal value of an additional agent transitions from positive to negative:*

$$P^* = \inf \left\{ P_{\text{SA}} \in [0, 1] : \frac{\partial}{\partial k} S_{\text{eff}}(k, P_{\text{SA}})\Big|_{k=1} \leq 0 \right\} \tag{5}$$

Conjecture 3.2 (Saturation Crossover)**.** *[Conjecture] For fixed topology t and error model, there exists $P^*(t) \in (0, 1)$ such that for $P_{\text{SA}} < P^*$, the optimal agent count $k^* > 1$, and for $P_{\text{SA}} > P^*$, $k^* = 1$. We term this a crossover rather than a phase transition: the regression model of Kim et al. (2025) yields a smooth interaction coefficient ($\beta = -0.408$), indicating a gradual shift rather than a sharp discontinuity in the physics sense. The crossover may sharpen as coordination overhead increases, but we lack evidence for a true singularity.*

Kim et al. (2025) report $P^* \approx 0.45$ from their regression model ($\beta_{P_{\text{SA}} \times \log(1+n_a)} = -0.408$, $p < 0.001$). Above this threshold, $\partial P / \partial n_a < 0$; adding agents actively degrades performance. This threshold was derived from general benchmarks (web navigation, quantitative reasoning, planning, tool use), not from code-generation tasks specifically, and may shift for software engineering where coupling is tighter and correctness requirements stricter.

3.4. Extended Amdahl's Law with Reliability Factor

Theorem 3.3 (Extended Amdahl's Law)**.** *[Synthesis] For a task with serial fraction s, n agents, effective per-agent error rate $\delta = \epsilon / f(t)$ (where $f(t) \geq 1$ is the coordination benefit factor), the effective speedup is:*

$$S_{\text{eff}}(n) = \frac{1}{s + \frac{1-s}{n}} \cdot (1 - \delta)^n \tag{6}$$

The first factor is classical Amdahl speedup (Amdahl, 1967); the second is the reliability factor.

Theorem 3.4 (Optimal Agent Count)**.** *[Synthesis] For $S_{\text{eff}}(n)$ as in (6) with $\delta \in (0, 1)$, there exists a unique $n^* \geq 1$ maximizing S_{eff}. For small serial fraction s:*

$$n^* \approx \frac{1}{\sqrt{\delta}} \tag{7}$$

Proof sketch. Write $\ln S_{\text{eff}} = -\ln(s + (1-s)/n) + n\ln(1-\delta)$. Differentiating and setting to zero: $(1-s)/(n^2 s + n(1-s)) = -\ln(1-\delta) \approx \delta$. For small s, this simplifies to $1/n^2 \approx \delta$, giving $n^* \approx 1/\sqrt{\delta}$. The maximum is unique because $\ln S_{\text{eff}}$ is strictly concave (the sum of a concave function and a linear function with negative slope). □

Corollary 3.5. *[Synthesis] For independent agents ($\delta = \epsilon$) with $\epsilon = 0.2$: $n^* \approx 2.2$. For centralized coordination ($\delta = \epsilon / f(\text{central})$) with $f \approx 3.9$ (from Kim et al.): $n^* \approx 4.4$. Coordination roughly doubles the optimal team size.*

3.5. Incorporating the Universal Scalability Law

Combining Gunther's Universal Scalability Law (Gunther, 2008) with the reliability factor:

$$S_{\text{eff}}(n) = \frac{n}{1 + \sigma(n-1) + \kappa\, n(n-1)} \cdot (1-\delta)^n \tag{8}$$

where σ is the serialization coefficient and κ is the coherency (crosstalk) penalty. The USL's κ term is quadratic in n, producing a characteristic "retrograde" curve where throughput decreases beyond an optimal point. With the additional exponential decay from $(1-\delta)^n$, the optimal n^* is pushed even lower. The three penalties (serialization, crosstalk, unreliability) explain why empirical optimal team sizes are small: 3 to 5 agents per practitioner consensus (Osmani, 2025), consistent with $n^* \approx 3$-4 from the model at typical parameter values.

4. Task Decomposition as Balanced Min-Cut

4.1. Formal Problem

Definition 4.1 (Task Decomposition)**.** *[Established] + [Synthesis] A task decomposition for k agents on coupling graph $G = (F, E, w)$ is a partition $\pi = \{T_1, \ldots, T_k\}$ of F solving:*

$$\min_{\pi} \sum_{\substack{(u,v)\in E \\ u\in T_i, v\in T_j \\ i\neq j}} w(u,v) \qquad \textit{subject to} \qquad |T_i| \le (1+\varepsilon)\cdot \frac{|F|}{k} \quad \forall i \tag{9}$$

where $\varepsilon > 0$ is the allowed imbalance.

This is the balanced min-cut problem.

4.2. NP-Hardness and Approximation

Theorem 4.1 (NP-Hardness)**.** *[Established] The $(k, 1)$-balanced partition problem (exactly equal parts) admits no polynomial-time approximation with any finite factor unless $P = NP$. Even for planar graphs and grids, balanced partitioning is NP-hard to approximate within any satisfactory ratio.*

This is proven by reduction from 3-partition, which is NP-hard in the strong sense.

When the balance constraint is relaxed ($\varepsilon > 0$), approximation becomes possible:

- Andreev and Räcke (2006): for any constant $\nu > 1$, $O(\log^{1.5} n)$ approximation ratio with (k, ν)-balanced partition.
- Arora et al. (2009): $O(\sqrt{\log n})$ approximation for sparsest cut via semidefinite programming.

For a codebase of 10,000 files, the $O(\log^{1.5} n)$ bound gives an approximation factor of roughly 46; the $O(\sqrt{\log n})$ bound gives roughly 3.2. These worst-case bounds motivate the use of heuristic methods that empirically perform much better.

4.3. Cheeger Inequality and Spectral Methods

Theorem 4.2 (Discrete Cheeger Inequality). *[Established] For graph G with normalized Laplacian eigenvalues $0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$, the minimum conductance $\phi(G)$ satisfies:*

$$\frac{\lambda_2}{2} \leq \phi(G) \leq \sqrt{2\lambda_2} \tag{10}$$

The spectral bisection algorithm (partition by sign of the Fiedler vector v_2 (Fiedler, 1973)) achieves a cut with conductance at most $\sqrt{2\lambda_2}$.

For k-way partitioning, Lee et al. (2014) proved higher-order Cheeger inequalities: $\phi_k(G) = O(k) \cdot \lambda_2 / \sqrt{\lambda_k}$, giving constant-factor approximations when there are few well-separated clusters.

4.4. Practical Algorithms

The Kernighan-Lin (KL) algorithm (Kernighan and Lin, 1970) is an iterative improvement heuristic for graph bisection running in $O(n^2 \log n)$ per pass, with no approximation guarantees but empirically within 5 to 15% of optimal. The Fiduccia-Mattheyses (FM) algorithm (Fiduccia and Mattheyses, 1982) improves on KL in two ways: $O(p)$ per pass (where p is total pins) via bucket priority queues, and native hypergraph support, which is the natural model for software systems where a shared interface creates a hyperedge connecting all consumers.

METIS (Karypis and Kumar, 1998) uses a three-phase multilevel approach (coarsening, initial partitioning, uncoarsening with refinement) achieving $O(|E|)$ runtime. It produces edge cuts consistently 10 to 50% better than spectral partitioning and is the practical tool of choice for large software dependency graphs.

4.5. Hypergraph Model for Software

In a standard graph model, if file X is used by files A, B, C, D, this creates four separate edges. In a hypergraph model, $\{X, A, B, C, D\}$ form a single hyperedge whose cut cost reflects that modifying X affects all consumers simultaneously. This distinction matters for agent coordination: if one agent modifies a shared interface, all agents consuming that interface must reconcile, regardless of how many individual edges are cut. The FM algorithm and hMETIS handle hypergraphs natively.

4.6. Cut-Conflict Bridge

Lemma 4.3 (Cut-Conflict Bridge (Modeling Assumption)). *[Synthesis] Under the assumption that the composite coupling metric w is calibrated against historical conflict data (so that conflict probability is approximately proportional to coupling weight), the pairwise conflict probability between agents i and j is bounded by:*

$$P(C_{ij}^{\text{dep}} \cup C_{ij}^{\text{sem}}) \leq \alpha \cdot \sum_{\substack{(u,v) \in E \\ u \in T_i, v \in T_j}} w(u, v) \tag{11}$$

for some constant $\alpha > 0$ that depends on the coupling-to-conflict conversion rate. Therefore, the total expected conflict count is bounded by:

$$\mathbb{E}[N_{\text{conflicts}}] \leq \alpha \cdot W_{\text{cut}}(\pi) \tag{12}$$

where $W_{\text{cut}}(\pi)$ is the total cut weight of partition π.

Proof sketch. Each cross-partition edge (u, v) with weight $w(u, v)$ represents coupling that may cause a conflict. The probability of a conflict arising from this edge is at most proportional to the coupling weight, by construction of the composite coupling metric (which is calibrated against historical conflict data (Kasi and Sarma, 2013)). Summing over all cut edges yields the bound. We emphasize that this lemma is a modeling assumption, not a deep theorem: it states that conflicts are bounded by a metric designed to predict conflicts. Its value lies in connecting the graph-partitioning literature (which minimizes cut weight) to the coordination problem (which minimizes conflicts). The constant α is not universal; it depends on the calibration of w and is empirically in the range 0.01 to 0.1 for human-authored code, though this has not been validated for agent-generated code specifically (see Section 11). Without a calibrated α, Theorem 4.4 provides a relative bound (better partitions yield proportionally fewer conflicts) rather than an absolute one. □

Theorem 4.4 (Decomposition Quality Bounds Conflict Rate). *[Synthesis] Let π^* be the optimal balanced k-partition and $\hat{\pi}$ be the partition from a spectral or METIS-based algorithm with approximation ratio β. Then:*

$$\mathbb{E}[N_{\text{conflicts}}(\hat{\pi})] \leq \beta \cdot \mathbb{E}[N_{\text{conflicts}}(\pi^*)] \tag{13}$$

For spectral bisection, $\beta = O(\sqrt{\log n})$. For METIS, β has no worst-case guarantee but empirically $\beta \approx 1.1$-1.5.

This closes the loop: partition quality directly determines merge conflict rate, and approximation guarantees translate into conflict rate guarantees.

5. Merge Conflict Probability Model

5.1. Five-Level Taxonomy

We adopt a five-level conflict taxonomy, synthesized from the software merging literature (Accioly et al., 2018; Apel et al., 2012; Falleri et al., 2014; Sousa et al., 2018):

1. **Textual**: overlapping line-level changes in the same file. Detected by standard three-way merge (git).
2. **Structural**: AST-level conflicts (e.g., one agent moves a method while another modifies its body). Detected by structured merge tools such as JDime (Apel et al., 2012), which reduce reported conflicts by 39% compared to textual merge.
3. **Dependency**: conflicting package additions or import changes. Detected by build-level analysis.
4. **API**: signature changes that break callers (one agent changes a function signature while another adds calls to the old signature). Detected by refactoring-aware tools such as IntelliMerge (Shen et al., 2019), achieving 88.48% precision and 90.22% recall.
5. **Semantic**: behaviorally incompatible changes that merge cleanly at all syntactic levels. Verified by compositional analysis such as SafeMerge (Sousa et al., 2018), which verified correctness in 75% of benchmarks.

5.2. Probabilistic Graphical Model

Definition 5.1 (Multi-Level Conflict Model). *[Synthesis] Let k agents produce diffs $\Delta_1, \ldots, \Delta_k$. The conflict event at level $\ell \in \mathcal{L} = \{\text{text}, \text{struct}, \text{dep}, \text{API}, \text{sem}\}$ between agents i and j is C_{ij}^{ℓ}. The joint*

probability factors as:

$$P(C \mid \Delta_1, \ldots, \Delta_k) = \prod_{i<j} P(C_{ij} \mid \Delta_i, \Delta_j) \tag{14}$$

assuming pairwise conditional independence given the diffs. Each pairwise conflict probability decomposes across levels:

$$P(C_{ij} \mid \Delta_i, \Delta_j) = 1 - \prod_{\ell \in \mathcal{L}} \left(1 - P(C_{ij}^{\ell} \mid \Delta_i, \Delta_j)\right) \tag{15}$$

The per-level probabilities depend on overlap structure:

$$P(C_{ij}^{\text{text}}) = 1 - (1 - \rho_{ij}^{\text{file}})^m \qquad \text{(file overlap)} \tag{16}$$

$$P(C_{ij}^{\text{struct}}) = P(C_{ij}^{\text{text}}) \cdot p_{\text{struct}} \qquad \text{(conditional on textual overlap)} \tag{17}$$

$$P(C_{ij}^{\text{dep}}) = \rho_{ij}^{\text{dep}} \cdot p_{\text{break}} \qquad \text{(dependency coupling)} \tag{18}$$

$$P(C_{ij}^{\text{API}}) = \rho_{ij}^{\text{API}} \cdot p_{\text{sig}} \qquad \text{(API coupling)} \tag{19}$$

$$P(C_{ij}^{\text{sem}}) = \rho_{ij}^{\text{sem}} \cdot p_{\text{behav}} \qquad \text{(semantic coupling)} \tag{20}$$

5.3. Quadratic Growth

Lemma 5.1 (Quadratic Scaling). *[Established] + [Synthesis] The expected number of pairwise conflicts grows as $O(k^2)$:*

$$\mathbb{E}\left[\sum_{i<j} 1[C_{ij}]\right] = \binom{k}{2} \cdot \bar{p}_{\text{conflict}} \tag{21}$$

where $\bar{p}_{\text{conflict}}$ is the average pairwise conflict probability.

Proof. By linearity of expectation. Each of $\binom{k}{2}$ pairs has conflict probability $P(C_{ij})$. Averaging gives the result. □

For $k = 5$ agents and $\bar{p}_{\text{conflict}} = 0.15$ (consistent with the 7.6% to 19.3% range reported by Kasi and Sarma 2013), the expected conflict count is $10 \times 0.15 = 1.5$. For $k = 10$: $45 \times 0.15 = 6.75$. The quadratic growth makes naive scaling prohibitive.

5.4. Contract Reduction to $O(k)$

Theorem 5.2 (Ownership Reduces Conflict Order). *[Synthesis] Under exclusive file ownership contracts (Definition 2.4), the textual, structural, and API conflict probabilities drop to zero for all pairs. The total conflict probability reduces from $O(k^2)$ to:*

$$P(\text{conflict}) = O(k \cdot w_{\text{cut}}) \tag{22}$$

where w_{cut} is the inter-partition coupling from the task decomposition. For well-separated partitions ($w_{\text{cut}} \to 0$), this approaches $O(1)$.

Proof sketch. Exclusive file ownership ensures $\rho_{ij}^{\text{file}} = 0$ for all $i \neq j$, zeroing textual, structural, and API conflict probabilities. Remaining conflicts (dependency and semantic) occur only across partition boundaries, where coupling is bounded by w_{cut}. If the coupling graph has bounded degree d at partition boundaries, the number of conflict-prone pairs is $O(k \cdot d)$ rather than $O(k^2)$. □

Corollary 5.3 (Contract Value). *[Synthesis] The reduction from $O(k^2)$ to $O(k)$ pairwise conflicts is the formal justification for file ownership contracts. The $26\times$ bug multiplier for manually-resolved conflicts makes each prevented conflict extremely high-value.*

6. Coordination as Concurrency Control

6.1. The Database Analogy

The analogy between multi-agent code generation and database concurrency control is remarkably direct (Bernstein et al., 1987). The observation that git worktrees provide a form of snapshot isolation is well-known in the version control community; our contribution is the formal analysis of which anomalies remain and how contracts close the gap. Under pessimistic coordination (analogous to two-phase locking), agents acquire exclusive ownership of files before beginning work; no two agents concurrently modify overlapping resources. Under optimistic coordination (analogous to optimistic concurrency control), agents work in isolated environments (git worktrees as private workspaces), then attempt to merge; if conflicts are detected, the agent's work is rejected and must be retried.

The key difference is cost: database transactions are millisecond-scale; agent tasks are minute-scale. The retry cost of an aborted agent task (discarding expensive LLM inference) shifts the calculus: even rare conflicts make optimistic approaches less attractive unless conflict detection happens before expensive computation.

6.2. Write Skew as the Critical Anomaly

Theorem 6.1 (Write Skew under Snapshot Isolation). *[Synthesis] Under* Snapshot *isolation (git worktrees), write skew is the only permitted anomaly class, and it corresponds exactly to semantic merge conflicts.*

Proof sketch. Worktrees provide each agent a frozen, consistent snapshot at dispatch time, preventing dirty reads and non-repeatable reads. The snapshot is a complete copy, preventing phantoms. Write-write conflicts on the same file regions are detected by git's merge mechanism (textual conflict detection). The remaining anomaly is write skew: agent A_i modifies $f_a \in F_i$ based on reading $f_b \in F_j$, while A_j modifies f_b based on reading f_a. Both commits succeed (disjoint file sets), but the combined state is inconsistent. This is precisely a semantic conflict. □

Corollary 6.2. *[Synthesis] Contracts eliminate write skew for covered interfaces. If both f_a's dependence on f_b and f_b's dependence on f_a are captured as interface specifications in C, then the guarantee of interface preservation (G2 in Definition 2.4) prevents the inconsistency.*

6.3. Assume-Guarantee Contracts

Each agent A_i operates under a contract $(Asm_i, Guar_i)$ following the assume-guarantee paradigm (Benveniste et al., 2018; Henzinger et al., 2002; Meyer, 1992):

Assumptions Asm_i:

(A1) No other agent modifies any file in F_i: $\forall j \neq i,\ \mathrm{WriteSet}(A_j) \cap F_i = \emptyset$.
(A2) All interfaces referenced by A_i but owned by other agents remain stable.
(A3) The snapshot S_i provided to A_i at dispatch time is consistent (compiles, passes tests).

Guarantees $Guar_i$:

(G1) Agent A_i modifies only files in F_i: $\mathrm{WriteSet}(A_i) \subseteq F_i$.
(G2) For every interface I_j in files owned by A_i, the post-modification signature matches sig_j.
(G3) The modifications, applied to S_i, compile and pass all tests scoped to F_i.

6.4. Composition Theorem

Theorem 6.3 (Contract Composition)**.** *[Synthesis] If every agent A_i satisfies its contract (assuming Asm_i holds, A_i establishes $Guar_i$), then the composed system satisfies:*

1. *No write-write conflicts (from disjointness and G1).*
2. *All specified interfaces are preserved (from G2).*
3. *For languages with sound separate compilation (e.g., Java, Rust, Go), the merged codebase compiles (from G3 and interface preservation). For dynamically-typed languages (Python, JavaScript), the guarantee weakens to: the merged codebase compiles at all statically checkable call sites.*

Proof sketch, rely-guarantee reasoning following Jones (1983). Step 1. By disjointness of F_i and guarantee G1, each agent's write set is disjoint from every other agent's. This establishes the rely condition: the environment does not modify A_i's files, satisfying A1.

Step 2. By G2, each agent preserves all interfaces in its owned files. Since A2 requires that interfaces outside F_i remain stable, and G2 ensures this for every F_j, the circular rely-guarantee discharge holds: $Guar_j$ for $j \neq i$ collectively implies Asm_i.

Step 3. Each agent's modifications compile in isolation (G3) with all consumed interfaces preserved (A2, now discharged). Since the merged codebase preserves all interfaces, every call site remains type-consistent. By compositionality of type checking, the merged result type-checks. □

Remark. *This proof is modular: adding agent A_{k+1} with a fresh disjoint F_{k+1} requires only verifying A_{k+1}'s contract, not re-verifying existing agents. This is the key advantage of assume-guarantee reasoning over monolithic verification.*

6.5. Correctness Conditions

Definition 6.1 (Strong Correctness)**.** *[Synthesis] A multi-agent execution producing diffs $\Delta_1, \ldots, \Delta_k$ is **strongly correct** (serializable) if there exists a sequential ordering π such that applying $\Delta_{\pi(1)}, \ldots, \Delta_{\pi(k)}$ in sequence produces the same final state as the merged result.*

Definition 6.2 (Weak Correctness)**.** *[Synthesis] A multi-agent execution is **weakly correct** if the merged result: (W1) compiles without errors; (W2) passes all pre-existing tests plus new tests added by agents; (W3) preserves all interface specifications in C.*

Theorem 6.4 (Correctness Hierarchy)**.** *[Synthesis] Strong correctness implies weak correctness, but not conversely.*

Proof. Forward: any serializable execution compiles, passes tests, and preserves interfaces. Converse counterexample: two agents both add a utility function with the same name but different implementations; after conflict resolution (renaming one), the result compiles and passes tests but corresponds to no sequential execution. □

Theorem 6.5 (Achievability under Snapshot + Contracts)**.** *[Synthesis] Under* Snapshot *isolation with contract coverage γ and 5-level merge detection with semantic detection rate d_5, weak correctness*

holds with probability at least:

$$P(\text{weak correct}) \geq 1 - (1-\gamma) \cdot \rho \cdot \binom{k}{2} \cdot (1 - d_5) \tag{23}$$

Proof sketch. Snapshot isolation eliminates dirty reads, non-repeatable reads, and phantoms (Theorem 6.1). Contract coverage γ eliminates write skew for covered interfaces. For uncovered interfaces, each cross-boundary pair has coupling probability ρ and undetected-conflict probability $(1 - d_5)$. Union bound over $\binom{k}{2}$ pairs yields the result. □

Calibration. Using $\rho \approx 0.15$ (Kasi and Sarma, 2013), $d_5 \approx 0.75$ (Sousa et al., 2018), $k = 5$, $\gamma = 0.8$: $P(\text{weak correct}) \geq 1 - 0.2 \cdot 0.15 \cdot 10 \cdot 0.25 = 0.925$. A 92.5% probability of weak correctness, improvable by increasing γ or d_5.

7. Topology Selection

7.1. Formal Objective

Definition 7.1 (Topology Selection Problem). *[Synthesis] Given task parameters $(k, \rho, p_{\text{single}}, \tau)$, select:*

$$t^* = \arg\max_{t \in \mathcal{T}} \frac{P(\text{success} \mid t)}{T(t) \cdot C(t)} \tag{24}$$

where $P(\text{success})$ is the probability of correct output, $T(t)$ is wall-clock time, and $C(t)$ is total compute cost.

7.2. Per-Topology Expressions

For each topology, with $\epsilon = 1 - p_{\text{single}}$:

Independent. $P_{\text{indep}} = (1-\epsilon)^k \cdot (1-\rho)^{\binom{k}{2}}$; $T_{\text{indep}} = \tau_{\text{single}} + \tau_{\text{merge}}(k)$; $C_{\text{indep}} = k \cdot c_{\text{single}}$.

Centralized. $P_{\text{central}} = (1-\epsilon_{\text{coord}})^{k+1} \cdot (1-\rho/f_c)^{\binom{k}{2}}$; $T_{\text{central}} = \tau_{\text{plan}} + \tau_{\text{single}} + \tau_{\text{integrate}}$; $C_{\text{central}} = (k+1) \cdot c_{\text{single}} + k \cdot c_{\text{coord}}$.

Hierarchical. $P_{\text{hier}} = (1-\epsilon)^k \cdot (1-\epsilon_{\text{mgr}})^{\lceil k/b \rceil} \cdot (1-\rho/f_h)^{k \cdot d_{\text{avg}}}$; $T_{\text{hier}} = d \cdot \tau_{\text{coord}} + \tau_{\text{single}}$ where $d = \lceil \log_b k \rceil$; $C_{\text{hier}} = k \cdot c_{\text{single}} + \lceil k/b \rceil \cdot c_{\text{mgr}}$.

Isolated-then-Merge. $P_{\text{iso}} = (1-\epsilon)^k \cdot (1-\rho)^{\binom{k}{2}} \cdot p_{\text{resolve}}^{N_{\text{conflicts}}}$; $T_{\text{iso}} = \tau_{\text{single}} + \tau_{\text{merge}}(k)$; $C_{\text{iso}} = k \cdot c_{\text{single}} + c_{\text{merge}}(k)$.

7.3. Dominance Conditions

Proposition 7.1 (Topology Dominance). *[Synthesis] Under the above model:*

1. *Independent dominates when $\rho \approx 0$ and ϵ is small.*
2. *Centralized dominates when ρ is moderate-to-high and ϵ is moderate: the benefit f_c on conflict reduction outweighs overhead.*
3. *Hierarchical dominates when $k > 8$ and ρ is moderate: the centralized coordinator becomes a bottleneck.*

4. *Isolated-then-merge dominates when ρ is low, ϵ is high, and automated resolution rate p_{resolve} is high.*

More precisely, centralized beats independent when:

$$f_c \cdot \binom{k}{2} \cdot \rho > c_{\text{coord}}/c_{\text{single}} + \epsilon_{\text{coord}} \tag{25}$$

7.4. Conway's Law as Communication Graph Containment

Definition 7.2 (Communication and Required Graphs)**.** *[Synthesis] The **communication graph** $G_T = (\mathcal{A}, E_T)$ has an edge $(A_i, A_j) \in E_T$ iff agents A_i, A_j can exchange messages under topology T. The **required communication graph** $G_R = (\mathcal{A}, E_R)$ has $(A_i, A_j) \in E_R$ iff agents A_i, A_j manage modules with inter-module dependencies.*

Theorem 7.1 (Conway's Law, Graph-Theoretic Form)**.** *[Synthesis] If the required communication graph is not a subgraph of the topology communication graph ($E_R \not\subseteq E_T$), then for every edge $(A_i, A_j) \in E_R \setminus E_T$, the agents managing the dependent modules cannot coordinate, producing a semantic conflict with probability proportional to the coupling strength of the underlying module dependency.*

Definition 7.3 (Topology-Module Mismatch Cost)**.** *[Synthesis]*

$$\text{Mismatch}(\alpha, T, G_M) = \sum_{(A_i,A_j)\in E_R\setminus E_T} \sum_{\substack{(M_a,M_b)\in E_M \\ \alpha(M_a)=A_i,\alpha(M_b)=A_j}} w(M_a, M_b) \tag{26}$$

where $\alpha : \mathcal{M} \to \mathcal{A}$ is the module-to-agent assignment. Mismatch $= 0$ when $E_R \subseteq E_T$ (Conway-aligned).

Theorem 7.2 (Inverse Conway for Agents)**.** *[Synthesis] The assignment α^* minimizing mismatch for a given topology T is the solution to a constrained balanced min-cut: partition the module graph into k parts minimizing uncovered cross-partition edge weight.*

For the independent topology ($E_T = \emptyset$), mismatch equals total cross-partition coupling, reducing to standard balanced min-cut. For mesh topology (E_T is complete), mismatch is zero for any assignment, but communication overhead is $O(k^2)$. For isolated-then-merge with contracts at coverage γ, the effective mismatch is $(1-\gamma)$ times the full mismatch, since contracts act as static communication channels conveying interface information without runtime message passing.

8. Quality-Adjusted Speedup

8.1. Definition

Standard speedup $S(k) = \Theta_{\text{par}}(k)/\Theta_{\text{seq}}$ does not account for quality costs.

Definition 8.1 (Quality-Adjusted Speedup)**.** *[Synthesis]*

$$\text{QAS}(k) = S(k) \cdot \frac{1 - d(k)}{1 - d(1)} \tag{27}$$

where $d(k)$ is the defect rate with k agents and $d(1)$ is the single-agent defect rate.

Equivalently, defining sequential throughput $\Theta_{\text{seq}} = p_{\text{single}} \cdot q(1)/t_{\text{cycle}}$ and parallel throughput $\Theta_{\text{par}}(k) = k \cdot p_{\text{single}} \cdot R(k) \cdot q(k)/(t_{\text{cycle}} + t_{\text{merge}}(k))$:

$$\text{QAS}(k) = \frac{k \cdot R(k) \cdot q(k)}{q(1) \cdot (1 + t_{\text{merge}}(k)/t_{\text{cycle}})} \cdot \frac{1 - d(k)}{1 - d(1)} \tag{28}$$

8.2. Throughput Crossover Analysis

The crossover condition $\Theta_{\text{par}}(k) > \Theta_{\text{seq}}$ requires:

$$G(k) > 1 + M(k) \tag{29}$$

where $G(k) = k \cdot R(k) \cdot q(k)/q(1)$ is the quality-adjusted parallelism gain and $M(k) = t_{\text{merge}}(k)/t_{\text{cycle}}$ is the merge overhead ratio.

Sensitivity to conflict rate. For $k = 5$, uniform pairwise conflict probability p_c, and $q(k)/q(1) = 0.95$:

- At $p_c = 0.10$: $R(5) = 0.9^{10} = 0.349$, $G(5) = 5 \times 0.349 \times 0.95 = 1.66$.
- At $p_c = 0.20$: $R(5) = 0.8^{10} = 0.107$, $G(5) = 5 \times 0.107 \times 0.95 = 0.51$.

Five agents with 20% pairwise conflict probability are slower than one agent, *regardless of merge cost*.

Sensitivity to merge time. For $k = 3$, $p_c = 0.15$, $q(k)/q(1) = 0.95$: $R(3) = 0.85^3 = 0.614$, $G(3) = 3 \times 0.614 \times 0.95 = 1.75$. Crossover requires $M(3) < 0.75$: merge time must be less than 75% of cycle time.

8.3. Calibration against DORA Data

The DORA 2025 data (DORA Team, Google Cloud, 2025) provides a partial, *analogical* calibration. **Caveat:** DORA measures human developers assisted by AI tools, not autonomous multi-agent systems. We use it here as a proxy for the qualitative effect of increased code generation throughput on quality metrics, not as a direct measurement of multi-agent QAS. PRs merged increased by 98% ($S \approx 2.0$), while bug rate increased by 9%. If baseline per-PR defect rate is $d_0 = 0.05$:

$$\text{QAS}_{\text{DORA}} = 1.98 \times \frac{1 - 1.09 \cdot d_0}{1 - d_0} = 1.98 \times \frac{0.9455}{0.95} = 1.97 \tag{30}$$

QAS is close to raw speedup when per-PR quality is maintained. However, organizational delivery metrics remained flat, implying that review, integration, and testing bottlenecks absorb the throughput gains.

8.4. Calibration against Kim et al. Data

We calibrate QAS against Kim et al.'s data under the *worst-case assumption* that all system errors become delivered defects (no error filtering by testing or review). This conflates the error amplification factor A_e (probability of at least one agent error) with the defect-rate multiplier (fraction of delivered defective features), which are distinct quantities. The results below are therefore pessimistic upper bounds; a more refined model would account for the error detection rate of the test and review pipeline.

For the independent architecture ($A_e = 17.2$) with $d(1) = 0.20$: $d(k)_{\text{indep}} = \min(1, 17.2 \times 0.20) = 1.0$ (saturated under the worst-case assumption).

$$\text{QAS}_{\text{indep}} = S(k) \times \frac{1 - 1.0}{1 - 0.20} = 0 \tag{31}$$

Independent multi-agent systems with high error amplification produce zero quality-adjusted value.

For the centralized architecture ($A_e = 4.4$): $d(k)_{\text{central}} = \min(1, 4.4 \times 0.20) = 0.88$. $\text{QAS}_{\text{central}} = S(k) \times 0.12/0.80 = S(k) \times 0.15$. Even with the best coordination, speedup must exceed $1/0.15 = 6.67$ for QAS to reach 1.0.

8.5. Demonstration: QAS < 1

Proposition 8.1. *[Synthesis] For any topology t and agent count $k > 1$, if:*

$$A_e(t,k) \cdot d(1) \geq 1 - \frac{1 - d(1)}{S(k)} \tag{32}$$

then $\mathrm{QAS}(k) < 1$*: parallelism is net-negative.*

This condition is easily satisfied when $d(1)$ is moderate (0.15 to 0.30), A_e is large (4 to 17), and $S(k)$ is modest (2 to 3). The regime $\mathrm{QAS} < 1$ is not a corner case; it is the default outcome for poorly coordinated multi-agent systems.

9. Empirical Calibration and Design Principles

9.1. Calibration Tables

Table 1 collects error amplification parameters from the literature.

Table 1 | Error amplification parameters from empirical studies.

Parameter	Value	95% CI	Source	Conditions
A_e (Independent)	17.2×	[14.3, 20.1]	Kim et al. (2025)	No shared state
A_e (Centralized)	4.4×	[3.8, 5.0]	Kim et al. (2025)	Hub coordinator; 285% overhead
A_e (Decentralized)	7.8×	Not reported†	Kim et al. (2025)	Peer-to-peer; 263% overhead
A_e (Hybrid)	5.1×	Not reported†	Kim et al. (2025)	Combined; 515% overhead
Bug mult. (conflict code)	2×	NR	Lesenich et al. (2017)	Any merge conflict
Bug mult. (manual resolution)	26×	NR	Lesenich et al. (2017)	Manual intervention

†Calibrations without CIs are point estimates; interpret with caution.

Table 2 collects merge conflict rate parameters.

Table 2 | Merge conflict rate parameters from large-scale studies.

Parameter	Value	Source	Sample
Textual conflict rate	17-20% of merges	Brun et al. (2011); Ghiotto et al. (2018)	3.4M LOC; 2,731 projects
Project-level range	7.6-19.3%	Kasi and Sarma (2013)	Multiple OSS projects
Build conflict rate	1-10%	Brun et al. (2011)	9 projects
Trivial resolution	75%	Ghiotto et al. (2018)	2,731 Java projects

Table 3 collects throughput and productivity parameters.

9.2. Coupling Density Dynamics

Software coupling grows over time per Lehman's Laws (Lehman, 1980). Model coupling density $\rho(G_t)$ as a stochastic process:

$$\rho(G_{t+1}) = \rho(G_t) + \alpha \cdot \Delta F_t - \beta \cdot R_t + \xi_t \tag{33}$$

where $\alpha \approx 0.02$-0.05 edges/file/feature is the coupling growth rate, β is the refactoring reduction rate (typically $\beta < 0.5\alpha$ without active refactoring), and $\xi_t \sim \mathcal{N}(0, \sigma^2)$ is noise.

Table 3 | Throughput and productivity parameters.

Parameter	Value	Source	Conditions
Parallel throughput mult.	2.2×	M1-Parallel	Academic; accuracy preserved
Practitioner-reported	~3×	Osmani (2025)	3 agents vs. 1 generalist
PRs merged increase	+98%	DORA Team, Google Cloud (2025)	10,000+ devs; objective telemetry
Review time increase	+91%	DORA Team, Google Cloud (2025)	Downstream cost
PR size increase	+154%	DORA Team, Google Cloud (2025)	AI-generated code
Bug rate increase	+9%	DORA Team, Google Cloud (2025)	Absolute count
Optimal team size	3-5 agents	Osmani (2025)	Practitioner consensus

Definition 9.1 (Partition Half-Life). *[Synthesis] The **partition half-life** $\tau_{1/2}$ is the expected time before inter-partition coupling exceeds a critical threshold:*

$$\mathbb{E}[\tau_{1/2}] \approx \frac{\rho^*_{\text{inter}} - \rho_{\text{inter}}(t_0)}{\dot{\rho} \cdot p_{\text{cross}}} \tag{34}$$

where $p_{\text{cross}} = 1 - \gamma(\mathcal{P})$ is the fraction of new coupling crossing partition boundaries.

For a well-partitioned codebase ($\gamma = 0.8$, $\dot{\rho} = 0.01$ edges/file/week): $\tau_{1/2} \approx 200$ weeks. For a poorly-partitioned one ($\gamma = 0.4$): $\tau_{1/2} \approx 67$ weeks. Task decompositions must be recomputed on the order of months to years; METIS's $O(|E|)$ runtime makes re-partitioning computationally cheap.

9.3. Six Design Principles

The theoretical framework yields the following actionable design principles, each derived from a specific formal result:

- **P1. Check before scaling.** If single-agent accuracy exceeds the saturation threshold ($P^* \approx 0.45$, Conjecture 3.2), do not add agents. Invest in improving the single agent instead.
- **P2. Enforce structurally, not via prompts.** Prompt-based constraints fail approximately 72% of the time under adversarial conditions (AgentSpec Team, 2025). File ownership must be enforced via filesystem permissions, git hooks, or tool-call interception, not via system messages.
- **P3. Partition before parallelizing.** Run METIS or hypergraph partitioning on the coupling graph before dispatching agents (Section 4). The Cut-Conflict Bridge (Lemma 4.3) guarantees that better partitions produce fewer conflicts.
- **P4. Use contracts to reduce conflict scaling.** File-ownership contracts reduce conflict growth from $O(k^2)$ to $O(k)$ (Theorem 5.2). The residual semantic conflicts are bounded by $(1-\gamma) \cdot \rho \cdot \binom{k}{2}$.
- **P5. Target 3 to 5 agents per team.** The Extended Amdahl's Law (Theorem 3.4) gives $n^* \approx 1/\sqrt{\delta}$. At typical error rates ($\epsilon = 0.2$, $f = 3.9$), $n^* \approx 4.4$. Beyond this, use hierarchical nesting rather than flat scaling.
- **P6. Measure QAS, not throughput.** Raw throughput ignores quality costs. QAS < 1 (Proposition 8.1) means parallelism is net-negative. Monitor the defect ratio $d(k)/d(1)$ and halt scaling when it exceeds $1/S(k)$.

10. Related Work

Multi-agent scaling science. Kim et al. (2025) provide the most comprehensive empirical evaluation of multi-agent system scaling, establishing error amplification factors, capability saturation,

and communication scaling laws. Our work builds on their empirical findings by developing a formal framework that explains their observations and derives actionable design rules. Cemri et al. (2025) introduced MAST, the first empirically grounded failure taxonomy for multi-agent LLM systems, identifying 14 failure modes across 1,600+ traces.

AI-assisted development metrics. The DORA 2025 report (DORA Team, Google Cloud, 2025) provides the largest objective measurement of AI's impact on software engineering practice. The productivity paradox it documents (individual gains, organizational stagnation) motivates our QAS metric, which formalizes the quality cost of parallelism. McKinney (2026) argued that Brooks's Law (Frederick P. Brooks, 1975) applies directly to LLM agents, a position our Extended Amdahl's Law (Theorem 3.3) formalizes.

Merge conflict research. Brun et al. (2011) pioneered proactive conflict detection with Crystal. Kasi and Sarma (2013) developed Cassandra for conflict-minimizing task scheduling. Ghiotto et al. (2018) provided the largest study of merge conflict characteristics (2,731 Java projects). Accioly et al. (2018) identified structural conflict patterns. Sousa et al. (2018) formalized semantic conflict-freedom verification with SafeMerge. Zhu et al. (2024) introduced ConGra, finding that general-purpose LLMs outperform code-specialized ones at conflict resolution, a counterintuitive result relevant to coordination agent design.

Merge tools. Falleri et al. (2014) developed GumTree for fine-grained AST differencing. Shen et al. (2019) developed IntelliMerge for refactoring-aware merging (88.48% precision). Apel et al. (2012) developed JDime for semistructured merge (39% conflict reduction).

Graph partitioning. Fiedler (1973) introduced algebraic connectivity and spectral bisection. Kernighan and Lin (1970) developed the KL heuristic. Fiduccia and Mattheyses (1982) improved it with linear-time passes and hypergraph support. Karypis and Kumar (1998) developed METIS with $O(|E|)$ multilevel partitioning. Arora et al. (2009) achieved $O(\sqrt{\log n})$ sparsest cut via SDP. Andreev and Räcke (2006) provided $O(\log^{1.5} n)$ balanced partitioning. Lee et al. (2014) proved higher-order Cheeger inequalities.

Software modularity. Mitchell and Mancoridis (2006) developed Bunch for automatic software modularization. Murphy et al. (2001) developed reflexion models for comparing intended and actual architecture. Martin (1994) defined coupling metrics (afferent, efferent, instability). MacCormack et al. (2012) validated Conway's Law empirically via the mirroring hypothesis.

Distributed systems. Bernstein et al. (1987) established the foundations of concurrency control. Berenson et al. (1995) formalized isolation levels and identified write skew. Fekete et al. (2005) showed how to make snapshot isolation serializable via promotion, a technique analogous to our use of contracts to close the gap from snapshot to serializable isolation for agents. Herlihy and Wing (1990) defined linearizability as the gold standard for concurrent correctness; our strong correctness condition (Definition 8.1) is the agent analogue. Lamport (1978) defined happened-before and logical clocks. Herlihy (1991) proved the consensus number hierarchy. Fischer et al. (1985) proved the impossibility of deterministic consensus with one faulty process. Ongaro and Ousterhout (2014) developed Raft for understandable consensus.

Contract-based design. Meyer (1992) introduced Design by Contract. Henzinger et al. (2002) developed assume-guarantee methodology. Jones (1983) developed rely-guarantee reasoning. Benveniste et al. (2018) provided a comprehensive theory of contracts for system design.

Industrial systems. Cursor (2026) documented the evolution from flat locking to planner/worker/judge topology, providing the strongest empirical evidence that hierarchical role separation succeeds at scale. Osmani (2025) synthesized practitioner patterns including optimal team sizing and tier-based tool stacks.

Software evolution. Lehman (1980) established the laws of software evolution, particularly increasing complexity, which our coupling density dynamics model builds upon. Conway (1968) articulated the law relating organizational structure to system architecture.

11. Conclusion

11.1. Summary

We have developed a formal framework for reasoning about coordination architecture in multi-agent software engineering. The framework connects four bodies of theory (reliability engineering, graph partitioning, concurrency control, and assume-guarantee contracts) to the specific problem of coordinating LLM agents on shared codebases.

We summarize the framework via a **heuristic coordination model** that combines the three forces identified in this paper:

$$S_{\text{eff}} \approx \underbrace{\frac{n}{1+\sigma(n-1)+\kappa\, n(n-1)}}_{\text{USL throughput}} \cdot \underbrace{(1-\delta)^n}_{\text{reliability}} \cdot \underbrace{(1-\alpha\, W_{\text{cut}})}_{\text{integration success}} \tag{35}$$

This is a heuristic, not a derived theorem: the multiplicative combination assumes approximate independence of the three factors, which is violated when agent errors cause merge conflicts (coupling factors 2 and 3). The third factor requires $\alpha W_{\text{cut}} \in [0, 1]$, which must be verified for specific parameter values. Despite these limitations, the model captures the qualitative dynamics: parallelism gains (increasing in n), error accumulation (decreasing exponentially in n), and integration risk (decreasing with cut weight, which tends to grow with n). The optimal operating point is typically small ($n^* \approx 3$-5).

11.2. Proposed Verification Experiments

Five experiments would validate or falsify the key claims:

1. **Agent-specific conflict rate.** Run $k \in \{1, 2, 3, 5, 8\}$ agents on the same feature set across 10+ repositories. Measure textual, build, and semantic conflict rates; wall-clock merge time. This calibrates p_c, $t_{\text{merge}}(k)$, and $R(k)$.
2. **Direct QAS measurement.** For the same task set, run sequential and parallel configurations. Measure features/hour, defects/feature (within 48 hours), and review time. Minimum sample: 50 tasks per configuration for 20% effect detection at $\alpha = 0.05$.
3. **Coupling density evolution.** Instrument 5+ repositories with weekly coupling graph snapshots. Fit the stochastic evolution model (α, β, σ) over 6 months. Test partition half-life predictions.

4. **Error amplification on code tasks.** Replicate Kim et al.'s methodology on SWE-Bench Verified, HumanEval+, and MBPP+. Key hypothesis: A_e for code generation exceeds the general-benchmark values due to tighter coupling.
5. **Structural vs. prompt enforcement.** Compare prompt-based file ownership against structural enforcement (filesystem permissions) across 100+ agent executions. Measure violation rate, defect rate, and throughput.

11.3. Open Questions

Several important questions remain:

- Does the 45% saturation threshold hold for code generation specifically, or does the tighter coupling shift it lower?
- Can hierarchical topologies beat centralized at scale? Kim et al.'s "decentralized" category is coarse.
- What is the optimal merge strategy: LLM-based integration versus AST-level structured merge?
- How does iterative refinement (generate, test, fix loops) change the throughput calculus?
- Can the coupling-to-conflict conversion constant α (Lemma 4.3) be calibrated for agent-generated code?
- What is the long-term maintenance cost of multi-agent-generated code?

The framework developed here provides the formal scaffolding for answering these questions. The critical next step is empirical calibration: until the six unknown parameters identified in Section 9 are measured for agent-generated code specifically, the quantitative predictions remain approximate. The qualitative conclusions, however, are robust: small teams outperform large ones, structural enforcement dominates prompt-based constraints, partition quality determines conflict rate, and quality-adjusted speedup is the right metric for evaluating multi-agent coordination.

A. Extended Proofs

A.1. Proof of Theorem 3.1 (Error Amplification, Full Detail)

Proof. We prove three claims: the formula, strict monotonicity, and the asymptotic expansion.

Formula. Each of k independent agents succeeds with probability $1-\epsilon$. Joint success (all k correct) has probability $(1-\epsilon)^k$. System error probability is $1-(1-\epsilon)^k$. Dividing by single-agent error probability ϵ gives $A_e = (1-(1-\epsilon)^k)/\epsilon$.

Monotonicity. Treating k as a continuous variable and differentiating:

$$\frac{\partial A_e}{\partial k} = \frac{-(1-\epsilon)^k \ln(1-\epsilon)}{\epsilon}$$

Since $\ln(1-\epsilon) < 0$ for $\epsilon \in (0,1)$, the numerator is positive, so $\partial A_e/\partial k > 0$.

Expansion. Using the binomial series $(1-\epsilon)^k = \sum_{j=0}^{k} \binom{k}{j}(-\epsilon)^j$:

$$1-(1-\epsilon)^k = k\epsilon - \binom{k}{2}\epsilon^2 + \binom{k}{3}\epsilon^3 - \cdots \tag{36}$$

$$A_e = k - \binom{k}{2}\epsilon + \binom{k}{3}\epsilon^2 - \cdots \tag{37}$$

The leading correction term is $-\binom{k}{2}\epsilon < 0$, confirming $A_e < k$ for all $\epsilon > 0$. □

A.2. Proof of Theorem 3.4 (Optimal Agent Count, Full Detail)

Proof. Write $f(n) = \ln S_{\text{eff}}(n) = -\ln(s + (1-s)/n) + n\ln(1-\delta)$.

Differentiating:

$$f'(n) = \frac{(1-s)/n^2}{s + (1-s)/n} + \ln(1-\delta)$$

Simplifying the first term: $(1-s)/(n^2 s + n(1-s)) = (1-s)/(n(ns+1-s))$.

Setting $f'(n) = 0$:

$$\frac{1-s}{n(ns+1-s)} = -\ln(1-\delta) \approx \delta + \delta^2/2 + \cdots$$

For small s (small serial fraction), $ns + 1 - s \approx 1$, so $(1-s)/n \approx \delta$, giving $n \approx (1-s)/\delta \approx 1/\delta$. More precisely, the denominator is $n^2 s + n(1-s)$. For small s, the $n(1-s)$ term dominates until $n \gg (1-s)/s$, so the equation becomes $1/n^2 \approx \delta$, i.e., $n^* \approx 1/\sqrt{\delta}$.

Uniqueness. The second derivative:

$$f''(n) = -\frac{(1-s)(2ns+1-s)}{[n(ns+1-s)]^2}$$

For $s \in (0,1)$ and $n \geq 1$, $f''(n) < 0$, so f is strictly concave. A strictly concave function on $[1,\infty)$ has at most one critical point; since $f'(1) > 0$ (for small enough δ, adding a second agent helps) and $f'(n) \to \ln(1-\delta) < 0$ as $n \to \infty$, there is exactly one n^* where $f'(n^*) = 0$. □

A.3. Proof of Theorem 6.3 (Contract Composition, Full Detail)

Proof. We verify each claim by circular rely-guarantee discharge.

Claim 1: No write-write conflicts. Suppose for contradiction that files $f \in F_i$ and $f \in F_j$ with $i \neq j$ are both modified. By G1, $\text{WriteSet}(A_i) \subseteq F_i$ and $\text{WriteSet}(A_j) \subseteq F_j$. By the disjointness condition, $F_i \cap F_j = \emptyset$. So $f \in F_i$ and $f \in F_j$ is impossible. Contradiction.

Claim 2: Interface preservation. Fix interface $I_j \in \sigma(f)$ for some $f \in F_i$. By G2 for agent A_i, the post-modification signature of I_j matches sig_j. No other agent modifies f (Claim 1), so I_j's signature is preserved in the merged codebase. Since I_j was arbitrary, all interfaces are preserved.

Claim 3: Merged codebase compiles. By Claim 2, all interfaces are preserved. Each agent's modifications compile in isolation against the preserved interfaces (G3). We need to show the merged result also compiles.

Consider an arbitrary call site c in file $f \in F_i$ that calls interface I_j in file $g \in F_l$ with $l \neq i$. At dispatch time, c was type-correct against sig_j (A3: snapshot is consistent). Agent A_i may have modified f; any new call to I_j must be type-correct against sig_j (G3 ensures compilation against snapshot, which includes I_j with signature sig_j). Agent A_l preserves sig_j (Claim 2). Therefore c remains type-correct in the merged result.

Since all call sites remain type-correct and all agents' modifications individually compile, the merged result compiles. This step relies on compositionality of type checking, which holds for languages with sound separate compilation (Java, Rust, Go). For dynamically-typed languages (Python, JavaScript), the guarantee covers only statically checkable constraints; runtime type errors at uncovered call sites remain possible.

Modularity. Adding A_{k+1} with F_{k+1} disjoint from $\bigcup_{i=1}^{k} F_i$ preserves all three claims: Claim 1 extends because F_{k+1} is disjoint; Claim 2 extends because A_{k+1} preserves its own interfaces (G2); Claim 3 extends because A_{k+1}'s modifications compile against preserved interfaces. No existing agent's contract needs re-verification. □

A.4. Proof of Theorem 7.1 (Conway's Law, Full Detail)

Proof. Let $(A_i, A_j) \in E_R \setminus E_T$. By definition of E_R, there exist modules M_a and M_b with $\alpha(M_a) = A_i$, $\alpha(M_b) = A_j$, and $(M_a, M_b) \in E_M$ (inter-module dependency). By $(A_i, A_j) \notin E_T$, agents A_i and A_j cannot exchange messages.

Consider the case where A_i modifies the implementation of a function in M_a that M_b depends on. Agent A_j, responsible for M_b, cannot learn about this modification (no communication channel). If A_j writes code in M_b that depends on the pre-modification behavior of M_a, the result is a semantic conflict.

The probability of this conflict is proportional to: (a) the probability that A_i modifies the relevant interface, which is proportional to the coupling weight $w(M_a, M_b)$; and (b) the probability that A_j's modifications depend on the changed behavior. Under the assumption that modification probability is proportional to coupling weight and that agents cannot selectively avoid conflicting changes without communication, the conflict probability is $\Theta(w(M_a, M_b))$. □

B. Notation Table

References

P. Accioly, P. Borba, and G. Cavalcanti. Understanding semi-structured merge conflict characteristics in open-source Java projects. *Empirical Software Engineering*, 23(4):2051–2085, 2018. doi: 10.1007/s10664-017-9586-1.

AgentSpec Team. AgentSpec: Customizable runtime enforcement for AI agents. In *Proceedings of ICSE 2026*, 2025. arXiv:2503.18666.

G. M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the AFIPS Spring Joint Computer Conference*, pages 483–485. ACM, 1967.

K. Andreev and H. Räcke. Balanced graph partitioning. *Theory of Computing Systems*, 39(6):929–939, 2006. doi: 10.1007/s00224-006-1350-7.

S. Apel, J. Liebig, B. Brandl, C. Lengauer, and C. Kästner. Semistructured merge: Rethinking merge in revision control systems. In *Proceedings of ESEC/FSE 2011*, 2012. Also: Structured merge with auto-tuning, ASE 2012.

S. Arora, S. Rao, and U. Vazirani. Expander flows, geometric embeddings and graph partitioning. *Journal of the ACM*, 56(2):Article 5, 2009.

A. Benveniste, B. Caillaud, D. Nickovic, R. Passerone, J.-B. Raclet, P. Reinkemeier, A. Sangiovanni-Vincentelli, W. Damm, T. A. Henzinger, and K. G. Larsen. Contracts for system design. *Foundations and Trends in Electronic Design Automation*, 12(2–3):124–400, 2018.

H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, and P. O'Neil. A critique of ANSI SQL isolation levels. In *Proceedings of the 1995 ACM SIGMOD International Conference on Management of Data*, pages 1–10, 1995. doi: 10.1145/223784.223785.

Table 4 | Notation used throughout the paper.

Symbol	Meaning	Defined in
k	Number of agents	Def. 2.2
ϵ	Per-agent error rate $(1 - p_{\text{single}})$	Def. 2.2
$\mathcal{T}$	Set of coordination topologies	Def. 2.3
t	A specific topology in $\mathcal{T}$	Def. 2.3
$A_e(t, k)$	Error amplification factor	Def. 2.2
$G = (F, E, w)$	Coupling graph (files, edges, weights)	Def. 2.1
C	Coordination contract	Def. 2.4
F_i	Owned file set of agent A_i	Def. 2.4
γ	Contract coverage (fraction of covered call sites)	§6
$R(n)$	Reliability factor	Thm. 3.3
$S(n)$	Amdahl speedup	§3
$S_{\text{eff}}(n)$	Effective speedup (Amdahl × reliability)	Thm. 3.3
δ	Effective per-agent error under coordination	Thm. 3.3
$f(t)$	Coordination benefit factor	§3
n^*	Optimal agent count	Thm. 3.4
ρ	Average inter-task coupling density	Def. 5.1
σ	USL serialization coefficient	Eq. (8)
κ	USL coherency (crosstalk) coefficient	Eq. (8)
s	Serial fraction (Amdahl)	§3
$\phi(G)$	Graph conductance (sparsest cut ratio)	Thm. 4.2
λ_2	Fiedler value (algebraic connectivity)	Thm. 4.2
$W_{\text{cut}}(\pi)$	Total cut weight of partition π	Lem. 4.3
C_{ij}^{ℓ}	Conflict between agents i, j at level ℓ	Def. 5.1
P^*	Capability saturation threshold	Def. 3.2
$\text{QAS}(k)$	Quality-Adjusted Speedup	Def. 8.1
$d(k)$	Defect rate with k agents	Def. 8.1
G_T	Communication graph of topology T	§7
G_R	Required communication graph	§7
$\tau_{1/2}$	Partition half-life	§9
α (Lem. 4.3)	Coupling-to-conflict conversion constant	Lem. 4.3
α (§7)	Module-to-agent assignment function	§7
α (§9)	Coupling growth rate	§9

Note: α is overloaded across sections; context disambiguates.

P. A. Bernstein, V. Hadzilacos, and N. Goodman. *Concurrency Control and Recovery in Database Systems*. Addison-Wesley, 1987.

Y. Brun, R. Holmes, M. D. Ernst, and D. Notkin. Proactive detection of collaboration conflicts. In *Proceedings of ESEC/FSE*, pages 168–178, 2011.

M. Cemri, M. Z. Pan, S. Yang, et al. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*, 2025. MAST taxonomy; 1,600+ annotated traces across 7 frameworks.

M. E. Conway. How do committees invent? *Datamation*, 14(4):28–31, 1968.

Cursor. Scaling long-running autonomous coding. https://cursor.com/blog/scaling-agents, 2026. Planner/Worker/Judge topology; hundreds of concurrent agents for weeks.

DORA Team, Google Cloud. 2025 state of AI-assisted software development. https://dora.dev/research/2025/dora-report/, 2025. Survey of nearly 5,000 technology professionals; supplemented by Faros AI telemetry from 10,000+ developers.

J.-R. Falleri, F. Morandat, X. Blanc, M. Martinez, and M. Monperrus. Fine-grained and accurate source code differencing. In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE)*, 2014. doi: 10.1145/2642937.2642982.

A. Fekete, D. Liarokapis, E. O'Neil, P. O'Neil, and D. Shasha. Making snapshot isolation serializable. *ACM Transactions on Database Systems*, 30(2):492–528, 2005. doi: 10.1145/1071610.1071615.

C. M. Fiduccia and R. M. Mattheyses. A linear-time heuristic for improving network partitions. In *Proceedings of the 19th Design Automation Conference*, pages 175–181, 1982.

M. Fiedler. Algebraic connectivity of graphs. *Czechoslovak Mathematical Journal*, 23(98):298–305, 1973.

M. J. Fischer, N. A. Lynch, and M. S. Paterson. Impossibility of distributed consensus with one faulty process. *Journal of the ACM*, 32(2):374–382, 1985. doi: 10.1145/3149.214121.

J. Frederick P. Brooks. *The Mythical Man-Month: Essays on Software Engineering*. Addison-Wesley, 1975.

G. Ghiotto, L. Murta, M. Barber, and A. van der Hoek. On the nature of merge conflicts: A study of 2,731 open source Java projects hosted by GitHub. *IEEE Transactions on Software Engineering*, 2018. doi: 10.1109/TSE.2018.2871083.

N. J. Gunther. A general theory of computational scalability based on rational functions. *arXiv preprint arXiv:0808.1431*, 2008.

T. A. Henzinger, S. Qadeer, and S. K. Rajamani. You assume, we guarantee: Methodology and case studies. In *Proceedings of CAV 2002*, volume 2404 of *LNCS*, pages 440–451, 2002.

M. Herlihy. Wait-free synchronization. *ACM Transactions on Programming Languages and Systems*, 13(1):124–149, 1991. doi: 10.1145/114005.102808.

M. P. Herlihy and J. M. Wing. Linearizability: A correctness condition for concurrent objects. *ACM Transactions on Programming Languages and Systems*, 12(3):463–492, 1990. doi: 10.1145/78969.78972.

C. B. Jones. Specification and design of (parallel) programs. In *Proceedings of the IFIP Congress*, pages 321–332, 1983.

G. Karypis and V. Kumar. A fast and high quality multilevel scheme for partitioning irregular graphs. *SIAM Journal on Scientific Computing*, 20(1):359–392, 1998. doi: 10.1137/S1064827595287997.

B. K. Kasi and A. Sarma. Cassandra: Proactive conflict minimization through optimized task scheduling. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, pages 732–741, 2013.

B. W. Kernighan and S. Lin. An efficient heuristic procedure for partitioning graphs. *Bell System Technical Journal*, 49(2):291–307, 1970.

Y. Kim, K. Gu, C. Park, et al. Towards a science of scaling agent systems. *arXiv preprint arXiv:2512.08296*, 2025. 180 configurations across 5 architectures, 3 LLM families, 4 benchmarks.

L. Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978. doi: 10.1145/359545.359563.

J. R. Lee, S. O. Gharan, and L. Trevisan. Multiway spectral partitioning and higher-order Cheeger inequalities. *Journal of the ACM*, 61(6):Article 37, 2014. Extended abstract at STOC 2012.

M. M. Lehman. Programs, life cycles, and laws of software evolution. *Proceedings of the IEEE*, 68(9): 1060–1076, 1980.

S. Lesenich, S. Apel, and C. Kästner. Balancing precision and performance in structured merge. In *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 367–377, 2017. Reports 26x bug multiplier for manually-resolved merge conflicts.

A. MacCormack, C. Baldwin, and J. Rusnak. Exploring the duality between product and organizational architectures: A test of the mirroring hypothesis. *Research Policy*, 41(8):1309–1324, 2012.

R. C. Martin. OO design quality metrics: An analysis of dependencies. *Report on Object Analysis and Design*, 1(3), 1994.

W. McKinney. The mythical agent-month. https://www.oreilly.com/radar/the-mythical-agent-month/, 2026. O'Reilly Radar.

B. Meyer. Applying "Design by Contract". *Computer*, 25(10):40–51, 1992. doi: 10.1109/2.161279.

B. S. Mitchell and S. Mancoridis. On the automatic modularization of software systems using the Bunch tool. *IEEE Transactions on Software Engineering*, 32(3):193–208, 2006.

G. C. Murphy, D. Notkin, and K. Sullivan. Software reflexion models: Bridging the gap between design and implementation. *IEEE Transactions on Software Engineering*, 27(4):364–380, 2001.

D. Ongaro and J. Ousterhout. In search of an understandable consensus algorithm. In *Proceedings of the 2014 USENIX Annual Technical Conference*, pages 305–319, 2014.

A. Osmani. The code agent orchestra. https://addyosmani.com/blog/code-agent-orchestra/, 2025. Practitioner synthesis of multi-agent orchestration patterns.

B. Shen, W. Zhang, H. Zhao, G. Liang, Z. Jin, and Q. Wang. IntelliMerge: A refactoring-aware software merging technique. In *Proceedings of the ACM on Programming Languages (OOPSLA)*, 2019. doi: 10.1145/3360596.

M. Sousa, I. Dillig, and S. K. Lahiri. Verifying semantic conflict-freedom in three-way program merges. In *Proceedings of the ACM on Programming Languages (OOPSLA)*, page Article 165, 2018. arXiv:1802.06551.

Y. Zhu et al. CONGRA: Benchmarking automatic conflict resolution. *arXiv preprint arXiv:2409.14121*, 2024.